# Reviewer Pack — Discriminant Exponentiation Bounds SOS Refutation Degree for…

Entry 149197523066 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 04:26:57 UTC

## Discriminant Exponentiation Bounds SOS Refutation Degree for Random 3-SAT

**Entry ID**: `149197523066` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 04:26:57 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry (Discriminant Theory) **Field B** (complexity object): SOS Refutation Degree for 3-SAT Instances

**Statement**:

> For a random 3-SAT instance with n variables, the discriminant of its polynomial system (encoding clauses as degree-2 polynomials) satisfies disc(Φ) ≥ 2^{Ω(n)} if and only if the SOS refutation degree for Φ is Ω(√n).

**Rationale (proposer's reasoning)**:

> Discriminants capture the geometric complexity of solution sets; their exponential growth correlates with the SOS hierarchy's need for deeper certificates to certify infeasibility in highly constrained systems.

**Taxonomy category**: `SOS_HIERARCHY` (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `1a0f2e0a5e5030f9`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> default: >=80% seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.00 | UNCERTAIN | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.00 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.00 | UNCERTAIN | UNCERTAIN |

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| KARP_LIPTON | SAFE | 0.95 | SAFE | SAFE |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

### 5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3sat_instance(n):
        clauses = []
        for _ in range(2 * n):
            literals = [random.choice([f'x{i}', f'~x{i}']) for i in range(n)]
            random.shuffle(literals)
            clause = ' or '.join(literals)
            clauses.append(clause)
        return ' and '.join(clauses)

    def polynomial_system_from_3sat(instance):
        polynomials = []
        for clause in instance.split(' and '):
            terms = []
            for literal in clause.split(' or '):
                if literal.startswith('~'):
                    var = literal[1:]
                    coeff = -1
                else:
                    var = literal
                    coeff = 1
                terms.append(f'{coeff}*x{var}')
            polynomials.append('+'.join(terms))
        return '+'.join(polynomials)

    def discriminant(poly):
        # This is a simplified version of computing the discriminant for a polynomial system.
        # For simplicity, we assume the polynomial is linear in each variable and compute the dete
        # of the matrix formed by the coefficients.
        terms = poly.split('+')
        n = len(terms)
        A = [[0] * n for _ in range(n)]
        b = [0] * n
        for term in terms:
            if 'x' not in term:
```

```
            continue
        var = term[1:]
        coeff = int(term[0]) if term[0].isdigit() else 1
        A[int(var) - 1][int(var) - 1] += coeff ** 2
    det = determinant(A)
    return det

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += ((-1) ** j) * matrix[0][j] * determinant(submatrix)
    return det

def sos_refutation_degree(poly, max_depth=10):
    # This is a simplified version of computing the SOS refutation degree.
    # For simplicity, we assume the polynomial is linear in each variable and compute the maxi
    # of the moment matrix that can be used to refute the polynomial.
    terms = poly.split('+')
    n = len(terms)
    M = [[0] * (n + 1) for _ in range(n + 1)]
    for term in terms:
        if 'x' not in term:
            continue
        var = term[1:]
        coeff = int(term[0]) if term[0].isdigit() else 1
        M[int(var) - 1][n] += coeff ** 2
        M[n][int(var) - 1] += coeff ** 2
    for i in range(n):
        M[i][i] += 1
    rank = matrix_rank(M)
    return rank

def matrix_rank(matrix):
    # This is a simplified version of computing the rank of a matrix.
    # For simplicity, we assume the matrix is square and compute its determinant.
    n = len(matrix)
    if n == 1:
        return 1 if matrix[0][0] != 0 else 0
    det = determinant(matrix)
    return 1 if det != 0 else 0

def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def binomial_coefficient(n, k):
    return factorial(n) // (factorial(k) * factorial(n - k))
```

```
    def is_power_of_two(x):
        return x > 0 and (x & (x - 1)) == 0

    n = random.randint(5, 40)
    instance = generate_3sat_instance(n)
    poly = polynomial_system_from_3sat(instance)
    disc = discriminant(poly)
    ref_degree = sos_refutation_degree(poly)

    metric_name = "SOS Refutation Degree"
    metric_value = ref_degree
    instances_tested = 1
    conjecture_holds = False
    counterexample = ""

    if is_power_of_two(disc) and disc >= 2 ** (0.3 * n):
        if ref_degree >= 0.5 * math.sqrt(n):
            conjecture_holds = True
    elif not is_power_of_two(disc) or disc < 2 ** (0.3 * n):
        if ref_degree < 0.5 * math.sqrt(n):
            conjecture_holds = True

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 8

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "mapping_undefined"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_93eae6dd.py", line 147, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_93eae6dd.py", line 117, in run_trial
    disc = discriminant(poly)
           ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_93eae6dd.py", line 58, in discriminant
    A[int(var) - 1][int(var) - 1] += coeff ** 2
      ^^^^^^^^
ValueError: invalid literal for int() with base 10: '1*xx1'
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> Test crashed due to invalid literal conversion, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Debug discriminant function's variable parsing logic to handle clause encodings properly

## 11. Audit log (LLM calls)

**Total LLM calls**: 7

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | qwen3:8b | 0 | 0 | 69043 |
| 2 | preregistration | ollama_remote | qwen3:8b | 0 | 0 | 24216 |
| 3 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 20696 |
| 4 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 11026 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 17889 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 15925 |
| 7 | judge | ollama_remote | qwen3:8b | 0 | 0 | 18485 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 177279 ms total latency. Provider mix: {'ollama_remote': 7}

*(full prompt+response transcripts available in `research/audit/149197523066.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/149197523066.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/149197523066.tar.gz` (if generated)